

9XAI Hackathon Handbook

AI-Based Traffic Monitoring and Traffic Flow Forecasting

First-Site Full Stack Intelligence Build

1. Purpose of This Hackathon

This hackathon is designed to produce the first-site full stack intelligence build for AI-based traffic monitoring, incident detection, traffic flow forecasting, signal optimization support, and operator-facing visualization at a selected intersection in Amman.

The build is intended to cover the full first-site scope in a modular manner so that the same architecture, logic, and operating model can later be scaled to multiple intersections and locations. The challenge is therefore not to produce isolated models or partial experiments. The challenge is to build the first complete site-level intelligence stack that can serve as the repeatable blueprint for future expansion. This is fully aligned with the 9XAI Lighthouse Cohort model of producing “crack-the-code solution-builds” and “shippable proof-of-concept” prototypes during the deep-dive journey.

You are building the first fully scoped traffic intelligence site that can become the repeatable blueprint for multi-location scaling.

2. Core Mission

Teams are expected to design and build an end-to-end traffic intelligence system for one representative site that enables:

- real-time traffic monitoring
- real-time incident and abnormal traffic behavior detection
- short-term traffic flow forecasting
- signal timing adjustment recommendations for human operator decision support
- web-based visualization and analysis
- modular readiness for future scaling across multiple sites

3. What You Are Building

Each team must build a complete system with the following mandatory modules:

- Data Acquisition Layer
- Real-Time Incident Detection Module
- Traffic Flow Forecasting and Signal Optimization Support Module
- Visualization and Decision-Support Dashboard

- Data Storage and Event Logging Layer

No team should treat the challenge as only a video analytics problem, only a forecasting problem, or only a dashboard problem. The challenge is the whole integrated stack.

4. Three-Phase Build Logic

The hackathon is structured into three build phases.

Phase 1

Traffic Data Sandbox Build

Teams hack, build, or acquire the dummy training dataset and simulated feeds that mimic live traffic operations for the first site.

Phase 2

Crack-the-Code Architecture and Feasibility Build

Teams build the architecture, quick builds, technical tests, benchmarks, and risky-part validations needed to prove that the full scope is achievable.

Phase 3

First-Site Full Stack Intelligence Build

Teams integrate and deliver the whole first-site traffic intelligence system as one coherent working solution.

5. System Concept and Architecture

The intended system concept brings together multiple data inputs, an AI-based processing core, and an operator-facing visualization layer.

At a high level, the build should bring together:

- CCTV video streams for live visual traffic observation
- historical CCTV video for model tuning and calibration
- traffic detector counts for traffic flow analysis and forecasting
- signal timing logs for signal-phase-aware forecasting and recommendation logic
- an AI-based processing layer that performs real-time incident detection and traffic flow forecasting
- a storage and logging layer that captures event metadata, forecasting outputs, system logs, and performance indicators

- a dashboard layer that surfaces all outputs for human analysis and decision support

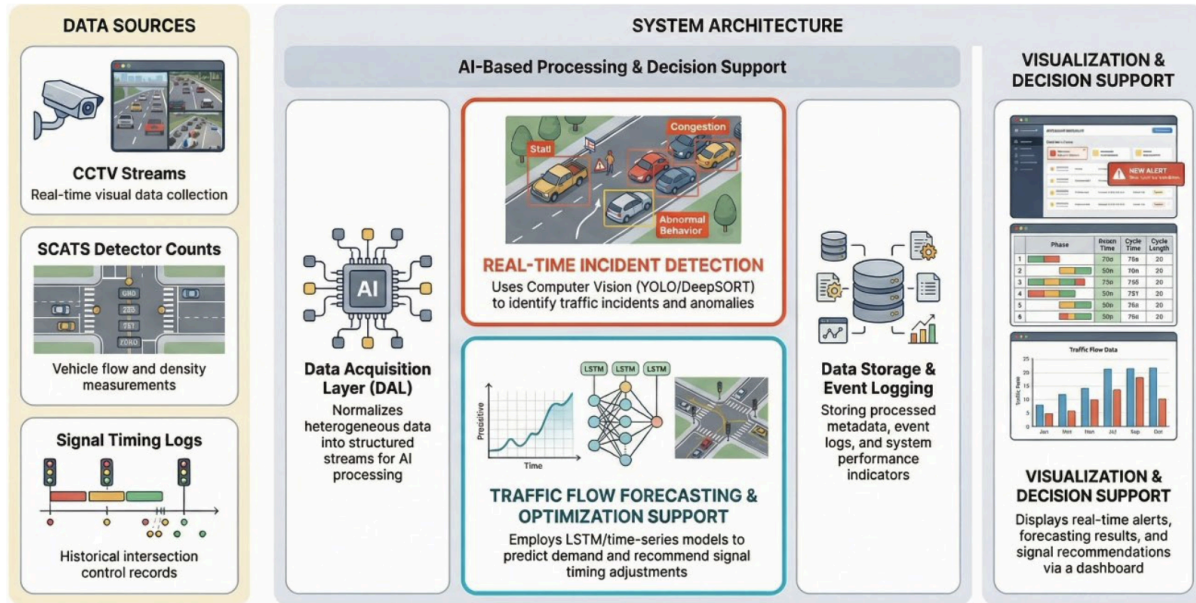


Figure A. High-level first-site traffic intelligence system architecture

6. Phase 1

Traffic Data Sandbox Build

Objective

To create a realistic and reusable traffic data sandbox that imitates the operating environment of the selected site and provides the foundation for model development, testing, validation, and integration.

Why this phase matters

The quality of the final system depends heavily on whether teams can work with data inputs that realistically mimic live operation. Phase 1 is not a side activity. It is a critical engineering layer. If this phase is weak, all downstream modules will be weak.

Required Phase 1 build scope

6.1 CCTV-like input environment

Teams must prepare a live-like video input setup that mimics the characteristics of the first site.

The expected assumptions are:

- one camera view for the target site
- RTSP-style streaming behavior
- H.264 or H.265 encoded stream assumptions
- video frame size of 1920 x 1080 RGB
- AI processing ingestion range of roughly 5 to 15 FPS

Teams may simulate the feed using replayed video, stream wrappers, or equivalent methods that create the feel of a live operational stream.

6.2 Historical CCTV training and calibration pack

Teams must prepare historical video samples that can be used for:

- model training
- model calibration
- AI tuning
- validation of event detection logic

The expected direction is approximately two weeks of representative video samples and selected clips that contain useful event scenarios for tuning and evaluation.

6.3 Traffic detector dataset

Teams must prepare a detector-count dataset that mirrors realistic traffic count exports.

The expected assumptions are:

- aggregated vehicle counts
- 15-minute time resolution
- 24-hour coverage
- approach or lane-based structure
- 22 detector assumptions across the site model, reflecting all approaching lanes plus extras where relevant
- The dataset should resemble exported detector logs rather than polished analytics tables.

6.4 Signal timing log dataset

Teams must prepare signal timing logs in a realistic event-log format.

Each line should include values such as:

- timestamp
- intersection ID
- phase number
- signal state such as GREEN ON, YELLOW ON, RED ON

This dataset is essential for signal-aware traffic forecasting and recommendation logic.

6.5 Intersection metadata pack

Teams must prepare a metadata file or schema describing the target site, including:

- camera location
- camera field of view
- lane configurations
- approach labeling
- stop line positions
- monitoring zones for queue spillback detection

6.6 Ground truth and annotation layer

Teams must create a labeled validation layer that supports training and evaluation of downstream AI modules.

This should include:

- vehicle labels where needed
- incident labels
- congestion-event labels
- abnormal stopping labels
- unexpected trajectory labels
- queue spillback markers
- selected event windows for validation

Phase 1 deliverables

- Traffic data sandbox package
- live-like video stream simulation or replay environment
- historical video training pack
- traffic detector dataset

- signal timing log dataset
- intersection metadata schema
- annotation and labeling package
- data dictionary
- methodology note explaining how the dummy data and simulated feeds were built or acquired

7. Phase 2

Crack-the-Code Architecture and Feasibility Build

Objective

To prove that the hardest parts of the first-site scope are technically feasible before final integration.

Why this phase matters

This is the de-risking phase. Teams are expected to build enough of the risky parts to prove that the whole system is achievable, not just to describe how it might work.

Required Phase 2 build scope

7.1 Architecture design

Each team must produce a clear modular architecture showing:

- all core system modules
- relationships between modules
- data flows between modules
- storage and logging layers
- dashboard interaction flows
- fault-handling paths
- system monitoring paths
- future scale considerations to additional sites

7.2 Data Acquisition Layer quick build

Teams must prove that they can:

- ingest live-like CCTV streams
- decode video frames
- ingest detector logs

- ingest signal timing logs
- normalize heterogeneous inputs into unified structured formats
- standardize timestamps
- map camera IDs
- map detector IDs
- normalize approach and lane labels
- validate corrupted or missing records
- flag invalid or incomplete data
- buffer or stream outputs for downstream AI modules

7.3 Real-Time Incident Detection quick build

Teams must prove feasibility of:

- vehicle detection using deep-learning object detection models such as YOLO-family models
- vehicle tracking using multi-object tracking methods such as DeepSORT or ByteTrack
- behavior analysis across frames
- detection of stalled vehicles
- detection of abnormal stopping
- detection of unexpected vehicle trajectories
- detection of queue spillback beyond designated zones
- detection of sudden congestion buildup
- classification of detected anomalies into meaningful event types such as possible traffic incident, congestion event, or abnormal traffic condition
- The output of this quick build should generate structured event notifications including:
 - timestamp
 - event type
 - location within the site
 - confidence score
 - snapshot image or short clip
 - estimated queue length or congestion indicator where feasible

7.4 Traffic Flow Forecasting quick build

Teams must prove feasibility of:

- training short-term traffic demand forecasting models
- using historical traffic counts per approach or detector
- using aggregated traffic volume data at 15-minute resolution
- using signal phase change logs
- using calendar indicators such as weekday or holiday markers
- predicting traffic demand 15 minutes ahead
- predicting traffic demand 30 minutes ahead
- predicting traffic demand 1 hour ahead
- optional extension to longer horizons where feasible

Models may include:

- Long Short-Term Memory (LSTM) networks
- temporal neural networks
- gradient boosting regression models
- other suitable time-series forecasting algorithms

7.5 Signal optimization support quick build

Teams must prove that they can generate recommendation outputs based on forecast demand, including:

- extension of green time for high-demand approaches
- reduction of green time during low-demand periods
- adjustments to cycle length
- identification of anticipated congestion periods
- This recommendation layer is strictly for decision support and human evaluation.

7.6 Dashboard quick build

Teams must prove the dashboard can meaningfully surface:

- real-time video monitoring with AI overlays
- incident alerts and event logs
- visualization of traffic flow patterns
- short-term traffic forecasts
- signal timing recommendations
- historical performance analysis
- system health indicators such as ingestion rate, dropped frames, and stream uptime

7.7 Security, read-only, and system-isolation proof

Teams must prove that the design remains:

- read-only toward source environments
- non-intrusive to operational infrastructure
- isolated from operational traffic signal control
- restricted to authorized users
- aligned with secure handling of traffic data and logs

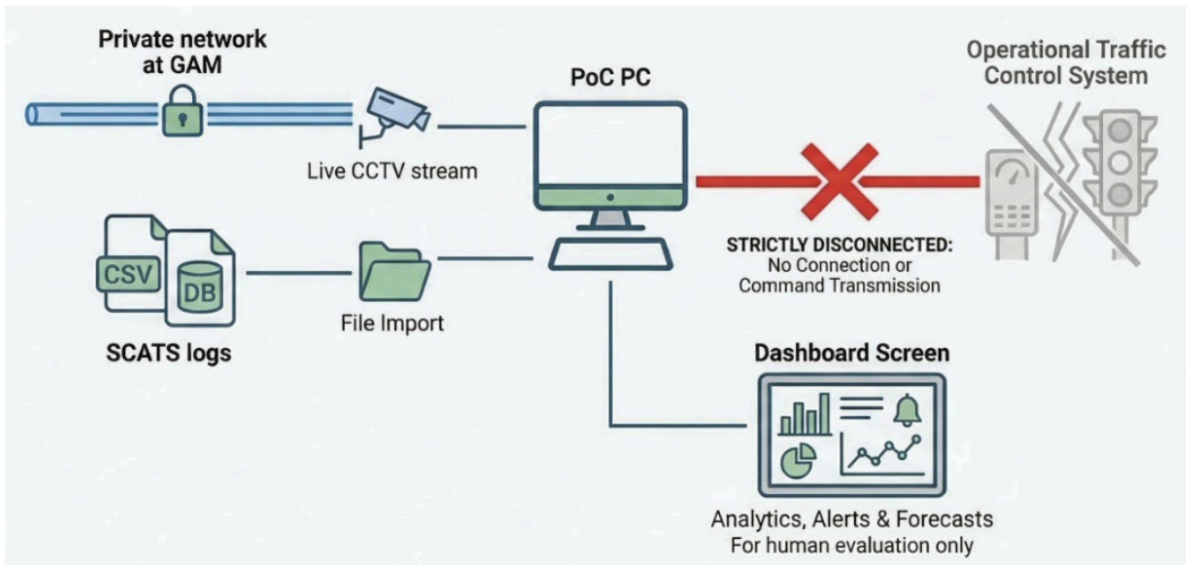


Figure B. Isolation model between the first-site traffic intelligence build and operational traffic control systems

Required benchmarks in Phase 2

Teams should benchmark at least:

- video ingestion stability
- frame decoding consistency
- event detection latency
- basic incident detection precision and recall
- forecasting performance against baseline methods
- dashboard responsiveness
- data-loss handling behavior
- stream reconnection behavior
- logging and monitoring completeness

Phase 2 deliverables

- architecture document
- system architecture diagram
- data flow diagram
- module interaction logic
- quick builds for risky parts
- technical benchmark report
- test cases
- validation notes
- risk register and mitigation plan
- security and isolation design note

- monitoring and fault-handling design note

8. Phase 3

First-Site Full Stack Intelligence Build

Objective

To build and integrate the complete first-site traffic intelligence system as one coherent working solution.

Required Phase 3 build scope

8.1 Data Acquisition Layer

The Data Acquisition Layer is the entry point for all system data and must securely collect, ingest, validate, normalize, and route traffic-related inputs for downstream use.

The Data Acquisition Layer must operate in read-only mode only. It must not modify source data, write to operational systems, or interfere in any way with live traffic signal control or operational traffic management infrastructure.

The layer must support:

- video stream ingestion
- batch ingestion for detector data and logs
- timestamp standardization
- camera ID mapping
- lane and approach labeling
- detector ID normalization
- unit conversion where required
- validation of missing frames, missing packets, corrupted files, or anomalous detector data
- flagging and optional discarding of invalid data
- streaming or buffered outputs for downstream AI modules

The layer should also include:

- automatic reconnection logic if a stream fails, with retry intervals in the range of 5 to 10 seconds
- monitoring of ingestion rate
- tracking of dropped frames
- tracking of stream uptime

8.2 Real-Time Incident Detection Module

This module must automatically detect traffic incidents and abnormal traffic behavior using computer vision methods applied to the CCTV feed.

The expected AI pipeline should include:

- vehicle detection
- vehicle tracking
- behavior analysis
- event detection and classification
- The system should detect or attempt to detect:
 - stalled vehicles
 - abnormal stopping
 - unexpected trajectories
 - queue spillback beyond designated zones
 - sudden congestion buildup
- The output should include:
 - timestamp
 - event type
 - location within the site
 - confidence score
 - snapshot image or short clip
 - estimated queue length or congestion indicator where possible

For storage efficiency, the system should prioritize event metadata storage, while storing short clips primarily around detected events for verification purposes.

8.3 Traffic Flow Forecasting and Signal Optimization Support Module

This module must forecast short-term traffic demand and generate recommendation outputs that support human evaluation of signal timing adjustments.

The forecasting logic should use:

- historical traffic counts per approach or detector
- aggregated traffic volume at 15-minute resolution
- signal phase change logs
- calendar information such as weekday or holiday indicators
- The module should generate predictions for:
 - 15 minutes ahead
 - 30 minutes ahead
 - 1 hour ahead
 - optional longer horizons where feasible

The recommendation logic should generate outputs such as:

- extension of green time for high-demand approaches

- reduction of green time during low-demand periods
- adjustments to cycle length
- identification of anticipated congestion periods

8.4 Visualization and Decision-Support Dashboard

The system must include a browser-based dashboard that enables operators and analysts to monitor outputs and interpret system behavior.

The dashboard should include:

- real-time video monitoring with AI detection overlays
- incident alerts
- event logs
- traffic flow pattern visualizations
- short-term traffic forecasts
- signal timing recommendations
- historical performance analysis
- system health indicators
- authorized-user access control

8.5 Data Storage and Event Logging Layer

The system must include a backend storage and logging layer that stores:

- processed traffic data
- event metadata
- forecasting results
- performance indicators
- system logs
- validation outputs
- monitoring outputs

This layer should be structured so that the system can later scale beyond one site without redesigning the entire data model.

Phase 3 deliverables

- integrated working system
- end-to-end walkthrough or demo
- full system design pack
- test and validation pack
- open-source component list
- database structure and design note

- reproducibility pack with analytics scripts and documentation of methods, formulas, and limitations
- user guidance material
- technical handover package
- final completion report including lessons learned and future scale recommendations

9. Technical Stack

Teams should adopt the following stack.

9.1 Video and stream handling

- Video ingestion: FFmpeg or GStreamer
- Stream decoding: OpenCV

9.2 Backend and ingestion services

- Python for ingestion services and backend orchestration

9.3 Messaging and orchestration

- Kafka or RabbitMQ for message queuing
- Airflow or Cron for scheduling and workflow orchestration

9.4 API layer

- REST or gRPC for service interfaces and integrations

9.5 AI and analytics stack

- PyTorch or TensorFlow for AI model development
- OpenCV for computer vision support
- Pandas and NumPy for data processing and analytics

9.6 Storage

- PostgreSQL or MySQL, or an equivalent relational database system, for structured storage of processed outputs, logs, metadata, and performance indicators

9.7 Frontend dashboard

- React or Vue.js, or equivalent, for the browser-based dashboard layer

9.8 Monitoring

- Prometheus for system monitoring and health metrics

Equivalent technologies may be accepted where justified, but teams should remain as close as practical to this technical direction.

10. Compute and Hardware Assumptions

- Teams should design the build with practical compute assumptions.
- Model training may use secure private cloud-based AI computing resources such as cloud GPU instances where needed. Training data must not be exposed through insecure services.
- For local inference and real-time processing, teams should assume:
 - a standard workstation-class PC
 - CPU-based processing where feasible
 - optionally a mid-range consumer-level GPU if needed for video inference
 - no dependency on high-end professional GPU infrastructure
- The design should minimize unnecessary infrastructure cost and complexity.

11. Technical Boundary Conditions

All teams must respect the following rules:

- The build must remain modular.
- The build must remain analytically isolated from live traffic signal control operations.
- The system must not transmit control commands to operational traffic management infrastructure.
- The outputs are for analysis, evaluation, and human decision support.
- The architecture must remain extendable to multiple sites in the future.
- The solution should prioritize open and stable technologies with no unnecessary long-term subscription dependency.

Read-only and operational isolation are mandatory. The system must operate as a standalone analytical environment. It must not be integrated with or connected to the operational traffic signal control system currently used at GAM. It must not write to operational systems, must not modify source data, and must not transmit any control commands to traffic signal controllers or to any operational traffic management equipment. All outputs are strictly for analysis, evaluation, and human decision support.

12. Team Design Expectations

Each team should ideally combine capabilities across:

- computer vision

- multi-object tracking
- time-series forecasting
- data engineering
- backend engineering
- frontend dashboard development
- systems architecture
- testing and validation
- product and operator workflow thinking

The strongest teams will not behave as isolated specialists. They will behave as end-to-end system builders.

13. Judging Criteria

A. Scope Coverage

How fully the team covers the whole first-site scope rather than only one subsystem

B. Architecture Quality

How strong, modular, extensible, and logically designed the system is

C. Sandbox Realism

How well the Phase 1 environment mirrors live traffic operation and supports downstream quality

D. Risk De-Risking Strength

How well Phase 2 proves that the hardest technical parts are feasible

E. AI Quality

How well the team handles detection, tracking, behavior analysis, forecasting, and recommendation logic

F. Dashboard Usefulness

How useful and understandable the outputs are for a human traffic operator or analyst

G. Reliability and Fault Handling

How well the system handles dropped feeds, missing data, corrupted data, and instability

H. Security and Isolation Discipline

How clearly the team preserves read-only behavior, system isolation, and controlled access

I. Reproducibility and Documentation

How easy the build is to understand, reproduce, and carry forward

J. Future Scale Readiness

How well the first-site build can later be extended to additional sites

14. Submission Requirements

Each team must submit:

- the traffic data sandbox package
- all code repositories
- system architecture diagram
- data flow diagram
- risk register
- benchmark report
- test cases and validation results
- database design and table structure
- list of open-source components used
- analytics scripts and documentation of methods, formulas, and limitations
- dashboard demo access or recorded walkthrough
- final presentation deck
- final technical handover pack

15. Demo Day Flow

Each team presentation should follow this order:

- site problem framing
- system concept and architecture
- Phase 1 traffic data sandbox build
- Phase 2 crack-the-code validations
- Phase 3 first-site full stack walkthrough
- live or recorded demo of major workflows
- benchmark highlights
- system limitations and lessons learned
- future scale pathway to multiple sites

16. What Success Looks Like

By the end of this hackathon, the strongest teams will have delivered:

- a realistic first-site traffic intelligence build
- a reusable traffic data sandbox
- a validated modular architecture
- proof that the most difficult technical parts are feasible
- an integrated working system across all mandatory modules
- a clear operator-facing dashboard
- a reproducible technical handover package
- a credible pathway for scaling the same design to multiple sites